**Plano de Garantia da Qualidade**

## 1. Objetivo

Este plano define como a equipe vai garantir a qualidade do Sistema de Folha de Pagamento durante todo o projeto: o padrão do código, o uso do Git, como as revisões são feitas, quais métricas vão ser acompanhadas e quando o sistema pode ser considerado pronto para homologação.

## 2. Padrões de Desenvolvimento

O código segue as convenções de código Java mais usadas no mercado, com base no Google Java Style Guide, e as regras abaixo:

| **Item** | **Padrão adotado** | **Exemplo** |
|---|---|---|
| Nome de classe | Substantivo, primeira letra de cada palavra maiúscula | ColaboradorComissionado |
| Nome de método | Verbo, primeira palavra minúscula e as outras com a primeira letra maiúscula | calcularSalario() |
| Nome de variável | Primeira palavra minúscula e as outras com a primeira letra maiúscula | salarioBase |
| Constante | Tudo maiúsculo, palavras separadas por _ | PERCENTUAL_MAXIMO |
| Idioma | Nomes em português, sem acento | quantidadeProduzida |
| Indentação | 4 espaços, uma instrução por linha | |
| Atributos | Sempre private, acessados por get e set | |
| Validação | Feita no construtor e nos métodos set, com mensagem de erro clara | |

| Item | Padrão adotado | Exemplo |
|---|---|---|
| Números fixos | Guardados em constante, nunca soltos no meio do código | 100 vira PERCENTUAL_MAXIMO |
| Comentários | Javadoc em toda classe e método público | |

### 3. Controle de Versão

O projeto usa Git, com o repositório no GitHub.

#### Branches

| Branch | Uso |
|---|---|
| main | Versão estável. Ninguém faz commit direto nela. |
| feature/nome | Nova funcionalidade. Ex.: feature/cadastro-colaborador |
| fix/nome | Correção de defeito. Ex.: fix/matricula-duplicada |
| docs/nome | Documentação. Ex.: docs/plano-de-testes |

Cada alteração é feita numa branch própria e só entra na main por pull request, depois de ser revisada pelo outro integrante.

#### Mensagens de commit

As mensagens começam com o tipo da alteração, seguido de uma descrição curta no presente:

| Tipo | Quando usar | Exemplo |
|---|---|---|
| feat | Nova funcionalidade | feat: adiciona exclusão de colaborador |
| fix | Correção de defeito | fix: impede matrícula duplicada |
| test | Testes | test: adiciona testes do cálculo de comissão |
| docs | Documentação | docs: adiciona plano de testes |

| Tipo | Quando usar | Exemplo |
|---|---|---|
| refactor | Melhoria no código sem mudar o comportamento | refactor: separa validação do construtor |

### 4. Revisões Técnicas

Toda alteração passa por revisão antes de entrar na main:

- **Código:** o integrante que não escreveu revisa o pull request usando o checklist abaixo. Se encontrar problema, comenta no pull request e a alteração só é aprovada depois de corrigida.
- **Documentos:** cada documento é revisado pelo outro integrante antes da entrega.
- **Revisão formal:** no fim da A1 e antes da entrega da A3, a equipe faz uma revisão geral e registra o resultado na Ata de Revisão Técnica, com data, participantes, problemas encontrados e correções.

**Checklist de revisão de código**

| Nº | Item verificado |
|---|---|
| 1 | O código compila e todos os testes passam. |
| 2 | Segue o padrão de nomes e formatação da seção 2. |
| 3 | Todas as entradas são validadas (RN002 a RN007, RN009, RN011 e RN012). |
| 4 | O programa não trava quando recebe um valor inválido. |
| 5 | Os cálculos batem com as fórmulas da ERS. |
| 6 | Toda alteração nova tem teste. |
| 7 | Não tem código repetido nem código comentado sem uso. |
| 8 | As mensagens mostradas ao usuário estão corretas e sem erro de digitação. |

## 5. Critérios de Qualidade

O sistema é considerado com qualidade quando:

- todos os requisitos da ERS têm pelo menos um caso de teste ligado a eles na matriz de rastreabilidade;
- o código segue o padrão da seção 2 e passou pela revisão;
- nenhum dado inválido é aceito e o sistema não fecha sozinho com entradas erradas;
- os cálculos de salário estão corretos para os três tipos de colaborador;
- os dados continuam salvos depois que o sistema é fechado e aberto de novo.

### Classificação dos defeitos

| Severidade | Descrição | Exemplo |
|---|---|---|
| Crítica | Sistema trava, perde dados ou calcula o salário errado | Comissão calculada sem dividir por 100 |
| Alta | Requisito não atendido, mas sem perda de dados | Não deixa excluir colaborador |
| Média | Falha com forma de contornar | Aceita nome só com espaços |
| Baixa | Problema de texto ou formatação | Erro de digitação numa mensagem |

## 6. Métricas e Metas

| Métrica | Meta | Como é medida |
|---|---|---|
| Cobertura de testes | 80% ou mais | Relatório de cobertura dos testes unitários |
| Defeitos críticos em aberto | 0 | Registro de Defeitos |
| Requisitos atendidos | 100% | Matriz de Rastreabilidade |

| Métrica | Meta | Como é medida |
|---|---|---|
| Casos de teste aprovados | 100% | Relatório de Testes |
| Defeitos encontrados e corrigidos | Todos registrados, todos os críticos e altos corrigidos | Registro de Defeitos |

## 7. Critérios de Aprovação

O sistema só é liberado para homologação quando:

- todas as metas da seção 6 forem atingidas;
- todos os casos de teste tiverem sido executados e aprovados;
- não houver nenhum defeito crítico ou alto em aberto;
- a revisão técnica final estiver registrada na ata;
- a matriz de rastreabilidade estiver atualizada, com todos os requisitos com status OK.